Universidade Federal de Lavras – UFLA/ICTIN
Engenharia de Software

Métricas em Engenharia de Software

Medição e Análise
Prof. Samira Santos da Silva

Introdução

- Existem três categorias principais:
  - Métricas de **Produto**
  - Métricas de **Processo**
  - Métricas de **Projeto**

Métricas de Produto

- **O que são?**
  - Medem o **software em si**.
  - Foco no **resultado final**.
- **Objetivo:**
  - Avaliar **qualidade** e **complexidade**.
- **Exemplos**:
  - Linhas de código (LOC).
  - Pontos de função.
  - Complexidade.

Métricas de Processo

- **O que são?**
  - Medem o **processo de desenvolvimento**.
- **Objetivo:**
  - Melhorar o processo.
- **Exemplos**:
  - Taxa de defeitos.
  - Tempo de correção.
  - Retrabalho.

Métricas de Projeto

- **O que são?**
  - Medem a **gestão do projeto**.
- **Objetivo:**
  - Planejar e controlar o projeto.
- **Exemplos**:
  - Custo;
  - Prazo;
  - Esforço;
  - Progresso.

Métricas - Resumo

Tipo	Mede	Foco
Produto	Software	Qualidade
Processo	Desenvolvimento	Melhoria
Projeto	Execução	Gestão

Métricas - Resumo

Métricas de Produto

- **Medidas quantitativas** usadas para descrever **características do software** em si (o produto).

Métricas de Produto

- Elas medem **o que o software é**, por exemplo:
  - Tamanho;
  - Complexidade;
  - Qualidade;
  - Estrutura.

- Focam no produto final ou artefatos.
- Diferente de:
  - **Métricas de processo** → como o software é desenvolvido.
  - **Métricas de projeto** → esforço, custo, prazo.

Por que medir o produto?

- **Entender melhor o sistema.**
  - Quantificar tamanho e estrutura.
  - Comparar sistemas diferentes.
  - Ter visão objetiva (não só percepção).

Por que medir o produto?

- **Estimar esforço e custo**
  - Sistemas maiores → mais esforço;
  - Sistemas complexos → mais tempo;
  - Base para planejamento.

Por que medir o produto?

- **Identificar problemas e riscos**
  - Alta complexidade → mais erros;
  - Muito código → difícil manutenção;
  - Baixa qualidade → risco de falhas.

Por que medir o produto?

- **Apoiar decisões técnicas**
  - Refatorar ou não?
  - Dividir módulos?
  - Reescrever parte do sistema?

Por que medir o produto?

- **Avaliar qualidade**
  - Código simples vs complexo;
  - Fácil vs difícil de manter;
  - Estruturado vs desorganizado.

Por que medir o produto?

O que **não é medido** não pode ser **gerenciado**.

Podemos medir tudo isso com **uma única métrica?**

Exemplos de Métricas de Produto

- Linhas de código (LOC);
- Pontos de função;
- Complexidade ciclomática;
- Número de módulos;
- Número de requisitos.

Linhas de código (LOC)

- O que é?
  - **LOC (Lines of Code)** é uma métrica que mede o tamanho do software pelo número de linhas de código.
- Tipos de contagem:
  - **Linhas físicas** → cada linha no arquivo (não importa o que tem na linha — só conta) .
  - **Linhas lógicas** → instruções reais que o programa executa (ex: comandos).

Linhas de código (LOC)

```
if (x > 0) {
    y = x + 1;
}
```

- Linhas físicas: 3
- Linhas lógicas: 1 ou 2 (depende da definição)

Linhas de código (LOC)

- Uma das primeiras métricas da engenharia de software.
- **Ponto forte**: Simplicidade → rápida aplicação.
- **Por que usar?**
  - Fácil de medir.
  - Pode ser automatizado.
  - Baixo custo de aplicação.
  - Amplamente utilizado na indústria.

Linhas de código (LOC)

- **Problemas principais:**
  - Depende da linguagem:
    - Java vs Python → LOC muito diferente.
  - Depende do estilo:
    - Código compacto vs expandido;
  - Não mede qualidade.
  - Não mede complexidade real.

Linhas de código (LOC)

- Exemplo:

Código A (Simples e Claro)

```
int somaPositivos(int[] nums) {
  int soma = 0;
  for (int n : nums) {
    if (n > 0) {
      soma += n;
    }
  return soma;
}
```

- LOC: 7 linhas
- Clareza: Alta
- Qualidade: Melhor

Código B (Verboso e Confuso)

```
int somaPositivos(int[] nums) {
  int soma = 0;
  int i = 0;
  while (true)) {
    if (i >= nums.length) {
      break;
    } else {
      int valorAtual = nums[i];
      if (valorAtual > 0) {
        soma = soma + valortual;
      } else {
        soma = soma + 0;
    }
  }
} return soma;
```

- LOC: 18 linhas
- Clareza: Baixa
- Qualidade: Pior

➡ Mais linhas de código não significa melhor software

Linhas de código (LOC)

- Exemplo:
  - LOC maior ≠ melhor.
  - LOC mede quantidade, não qualidade.
  - LOC pode até incentivar escrever mais código — o que é perigoso.

Linhas de código (LOC)

- **Aplicações:**
  - Estimar esforço:
    - Esforço ≈ LOC
  - Medir produtividade:
    - LOC / hora
  - Avaliar qualidade indireta:
    - Defeitos / LOC

Pontos de Função (PF)

- **O que são?**
  - **Pontos de Função** medem o tamanho do software com base nas funcionalidades entregues ao usuário.
- **Ideia central:**
  - Mede **o que o sistema faz**.
  - Não mede como ele é implementado.

Pontos de Função (PF)

- Exemplo:
  - Sistema bancário:
    - Fazer login
    - Consultar saldo
    - Transferir dinheiro
  - Cada funcionalidade contribui para o total de PF.

- Diferente do LOC, aqui ignoramos o código e olhamos o **valor entregue ao usuário**.

Pontos de Função (PF)

- **Tipos de elementos medidos:**
  - Entradas externas (EI):
    - Dados que entram no sistema para serem processados ou armazenados.
    - Ex: cadastro de usuário, envio de formulário.
  - Saídas externas (EO):
    - Dados que saem do sistema com processamento (cálculos, atualizações, etc.).
    - Ex: Relatórios, respostas, mensagens com resultados calculados.
  - Consultas (EQ):
    - Entrada + saída sem processamento significativo (apenas consulta), busca sem alteração.
    - Ex: Buscar dados e exibir na tela sem alterar nada.
  - Arquivos internos (ILF):
    - Conjunto de dados que o sistema mantém e gerencia internamente.
    - Ex: banco de dados de clientes do próprio sistema.
  - Interfaces externas (EIF):
    - Dados usados pelo sistema, mas que pertencem a outro sistema.
    - Ex: consumir dados de um sistema externo ou API.

Pontos de Função (PF)

- Cada componente:
  - Recebe um peso.
  - Contribui para o total de PF.

- O cálculo envolve classificar e contar esses elementos - por isso exige mais conhecimento.

Pontos de Função (PF)

- Como calcular:
  - Identificar funcionalidades;
  - Classificar (EI, EO, EQ, ILF, EIF);
  - Aplicar pesos;
  - Somar.

Pontos de Função (PF)

- Exemplo simples:

```
public void cadastrarUsuario(String nome) {
    salvarNoBanco(nome);
    System.out.println("Usuário cadastrado com sucesso!");
}
```

- Identificando os Pontos de Função
  - Entrada (EI)
    - nome (dado fornecido pelo usuário).
  - Arquivo Interno (ILF)
    - salvarNoBanco(nome).
  - Saída (EO)
    - Mensagem exibida ao usuários.

Pontos de Função (PF)

- Exemplo simples:

```
public void cadastrarUsuario(String nome) {
    salvarNoBanco(nome);
    System.out.println("Usuário cadastrado com sucesso!");
}
```

- Esse pequeno código contém:
  - 1 Entrada
  - 1 Arquivo Interno
  - 1 Saída

Pontos de Função (PF)

- Exemplo simples:

Tipo	Peso
EI (Entrada)	4
EO (Saída)	5
ILF (Arquivo Interno)	10

- Cálculo
  - EI: 1 × 4 = 4
  - EO: 1 × 5 = 5
  - ILF: 1 × 10 = 10
- Total de Pontos de Função:
  - PF=4+5+10=19
- Esse pequeno código: 19 PF.
- Mesmo sendo um **código pequeno**, ele já tem **várias funcionalidades**.
- Isso mostra que **Pontos de Função medem o valor entregue**, não o tamanho do código.

Pontos de Função (PF)

- **Vantagens dos Pontos de Função:**
  - Independente da linguagem;
  - Pode ser usado antes da implementação;
  - Foco no usuário;
  - Melhor para estimativas iniciais;
  - Impacto prático.

- Permite estimar:
  - Esforço;
  - Custo;
  - Prazo.

Pontos de Função (PF)

- **Limitações dos Pontos de Função (PF)**
  - Cálculo mais complexo.
    - Precisa identificar e classificar corretamente (EI, EO, EQ, ILF, EIF).
    - Pode gerar erros.
  - Requer conhecimento especializado.
  - Pode ser subjetivo.
    - Diferentes pessoas podem classificar de forma diferente.
    - Resultados variam.
  - Não considera qualidade do código.
    - Não considera:
      - Código ruim vs código bom.
      - Estrutura interna.
    - Só mede funcionalidade
  - Difícil em sistemas modernos (ex: mobile, tempo real, sistemas adaptativos, IA).
    - PF funcionam em para sistemas que processam dados, mas não para sistemas que interagem, reagem ou aprendem.

LOC vs Pontos de Função

- Diferença principal:
  - LOC → visão técnica
  - PF → visão do usuário

LOC	Pontos de Função
Mede código	Mede funcionalidade
Pós-implementação	Pré-implementação
Dependente da linguagem	Independente
Simples	Mais preciso

Atividade 5

```
int calcular(int x) {
    int resultado = 0;

    if (x > 0) {
        resultado = x * 2;
    } else {
        resultado = 0;
    }

    return resultado;
}
```

- Observe o código e responda:
  1) Quantas linhas físicas existem?
  2) Quantas linhas lógicas existem?
  3) Esse código é simples ou complexo? Justifique.
  4) O valor de LOC reflete bem a qualidade desse código?

Atividade 6

- Cenário:
  - Um sistema permite:
    - Usuário informa nome e email para cadastro
    - Sistema salva os dados
    - Sistema exibe mensagem de sucesso
    - Usuário pode consultar seus dados cadastrados
- Identifique:
  - Entradas (EI)
  - Saídas (EO)
  - Consultas (EQ)
  - Arquivos internos (ILF)

Tipo	Peso
EI	4
EO	5
EQ	4
ILF	10

- Calcule os PF usando os pesos.

Métricas de Tamanho

- LOC mede **quantidade de código**.

- Pontos de Função mede **tamanho funcional**.
  - Quanto funcionalmente o sistema entrega.

Métricas de Tamanho

- Além de **LOC e Pontos de Função**, existem métricas de tamanho baseadas em **classes, métodos ou módulos**.

- Exemplos:
  - Número de classes.
  - Número de métodos por classe.
  - Número de arquivos ou pacotes.

Métricas de Complexidade e Manutenibilidade

- Mas não basta ter pouco ou muito código.
- Precisamos medir **qualidade**, **complexidade** e **facilidade de manutenção**.
- É isso que as **métricas de complexidade e manutenibilidade** nos ajudam a entender.

Métricas de Complexidade

- Complexidade mede **quão difícil é entender, testar e modificar** um código.

- Diferente de LOC ou Pontos de Função, que medem quantidade ou tamanho funcional, a **complexidade foca na qualidade estrutural do software**.

- Um **código pequeno** pode ser **extremamente complexo** se tiver muitos **caminhos lógicos ou funções complicadas**, enquanto um código maior pode ser simples e bem organizado.

Métricas de Complexidade

- Exemplo (**pequeno e complexo** x **grande e simples**):

```python
def verifica_numero(n):
    if n > 0:
        if n % 2 == 0:
            return "Positivo e par"
        else:
            return "Positivo e ímpar"
    elif n < 0:
        if n % 2 == 0:
            return "Negativo e par"
        else:
            return "Negativo e ímpar"
    else:
        return "Zero"

print(verifica_numero(5))
print(verifica_numero(-4))
```

```python
def soma(a, b):
    return a + b

def subtrai(a, b):
    return a - b

def multiplica(a, b):
    return a * b

def divide(a, b):
    if b != 0:
        return a / b
    return "Divisão por zero"

print(soma(5, 3))
print(subtrai(5, 3))
print(multiplica(5, 3))
print(divide(5, 0))
```

Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - **Definição:**
    - Métrica da engenharia de software.
    - Desenvolvida por Thomas J. McCabe em 1976.
    - Usada para mensurar a complexidade de um módulo (classe, método, função etc.)
    - Baseia-se na contagem de caminhos independentes de execução
  - **Um caminho independente inclui pelo menos:**
    - Uma nova condição (desvio de fluxo) ou um novo conjunto de comandos executados.
  - **Indica quantos caminhos diferentes o código pode seguir até o final.**

Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - Existem **diferentes formas de calcular** a complexidade ciclomática:
    - Usando a notação de um **grafo de fluxo**.
    - Usando um **fluxograma**.
    - Por **análise estática** do código com ferramentas automatizadas.

Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - A partir de um grafo de fluxo ou fluxograma, há três fórmulas equivalentes:
    - **V(G) = R**
      - R = número de regiões do grafo
    - **V(G) = E – N + 2**
      - E = número de arestas (setas)
      - N = número de nós do grafo
    - **V(G) = P + 1**
      - P = número de nós-predicado
      - Válida quando cada nó-predicado tem no máximo duas saídas.

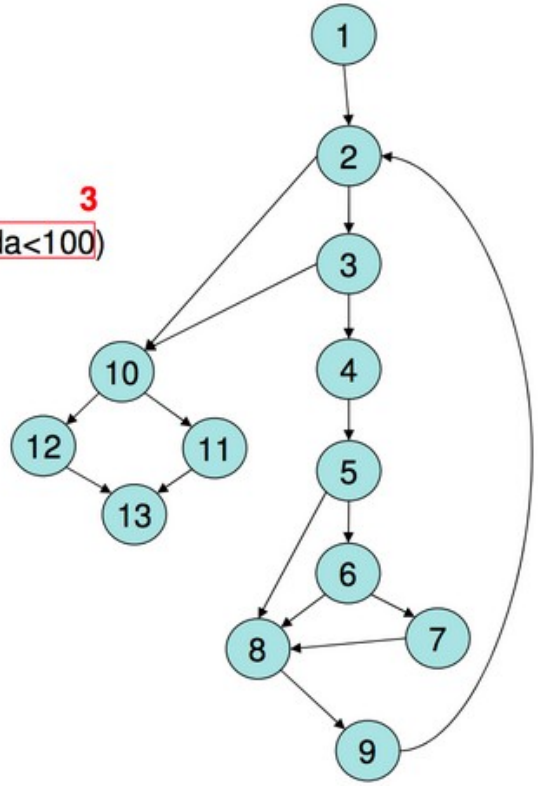
Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - **Nós-predicado**:
    - São pontos de decisão no código.
    - Podem desviar o fluxo de execução.
    - Exemplos: **if, while, switch**.

Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - **Exemplo** usando o **grafo de fluxo:**

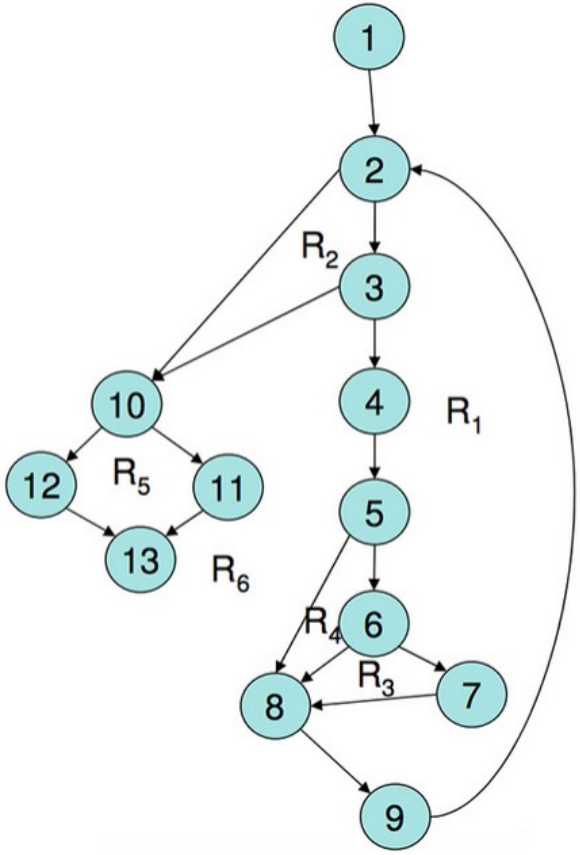
```
Procedimento média(valor[])
i=1;                                          1
soma=0;
total.entrada=0;
total.válidas=0
                                  2                    3
Faça-Enquanto (valor[i]≠-999 E total.entrada<100)
    incremente total.entrada de 1;  4
    Se 5 (valor[i]≥min 6 E valor[i]≤max)
    Então
7           incremente total.válidas de 1;
            soma = soma + valor[i]
    Fim-Se
8   incremente i de 1;
Fim-Enquanto  9
Se total.válidas>0  10
Então média = soma/total.válidas;  11
Senão média = -999;  12
Fim-Se  13
Fim média
```



Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**

  1) V(G) = R - onde R é o número de regiões do grafo de fluxo.



Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**

  2) V(G) = E – N + 2 onde E é o número de arestas (setas) e N é o número de nós do grafo G.

  - V(G) = 17 arestas/setas – 13 nós + 2 = 6

Métricas de Complexidade

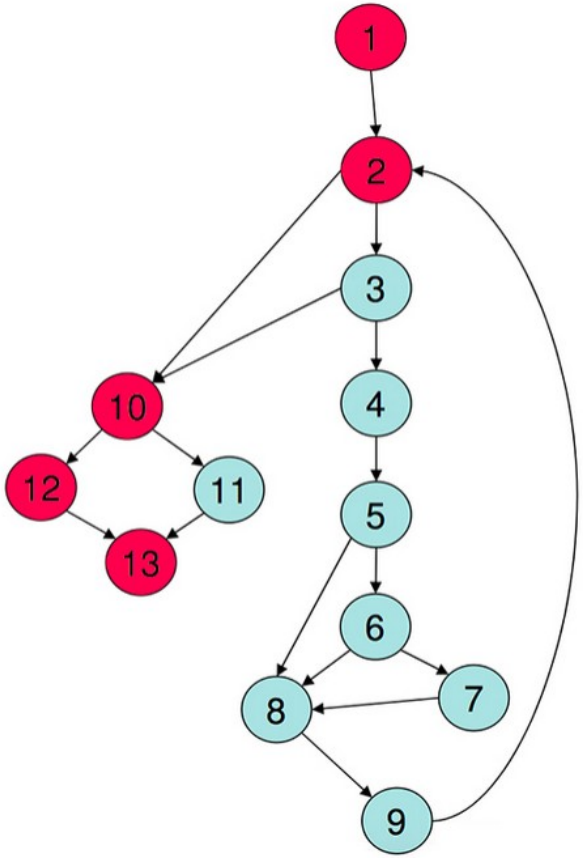
- **Complexidade Ciclomática (McCabe):**

  3) V(G) = P + 1 onde P é o número de nós-predicados contidos no grafo G.

  - V(G) = 5 nós-predicados + 1 = 6

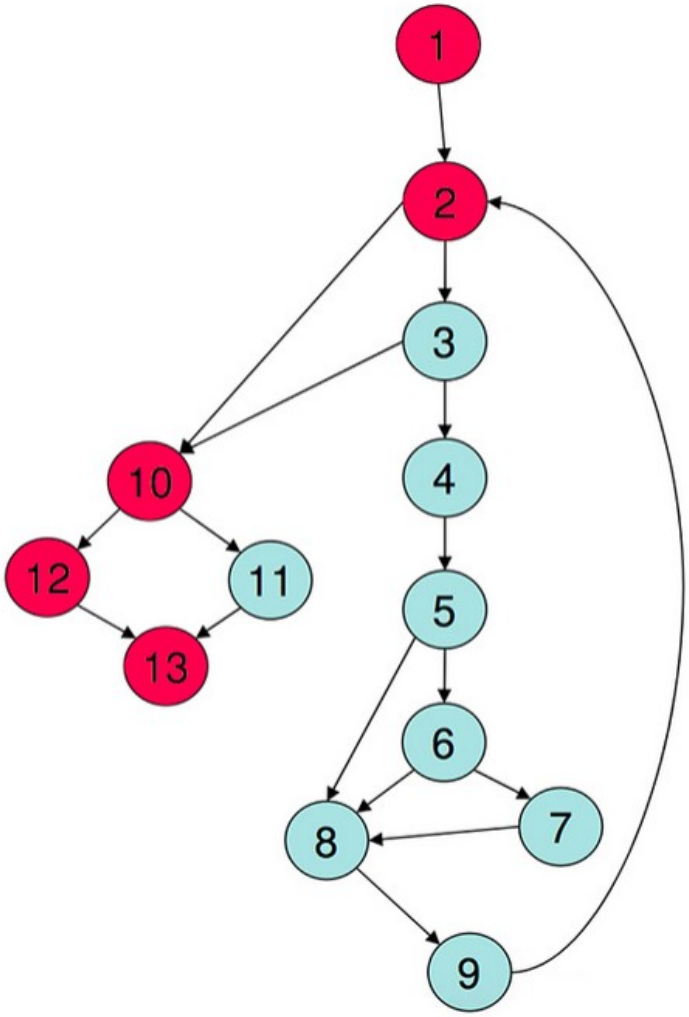
Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - No final temos, então, **6 caminhos independentes.**
    - **1) 1-2-10-12-13**
    - **2) 1-2-10-11-13**
    - **3) 1-2-3-10-11-13**
    - **4) 1-2-3-4-5-8-9-2-[…]**
    - **5) 1-2-3-4-5-6-8-9-2-[…]**
    - **6) 1-2-3-4-5-6-7-8-9-2-[…]**



Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - **Exemplo - Caminho independente 1:**



Métricas de Complexidade

- **Complexidade Ciclomática (McCabe):**
  - **Quais os parâmetros aceitáveis para a complexidade dos meus métodos?**
    - De acordo com o trabalho de McCabe, os **valores de referência** são:

Complexidade	Avaliação
1-10	Método simples. **Baixo risco**.
11-20	Método razoavelmente complexo. **Moderado risco**.
21-50	Método muito complexo. **Elevado risco**.
51-N	Método de **altíssimo risco** e bastante instável.

Atividade 8

- Considere o código:

```
int max(int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}
```

- Desenhe o grafo de fluxo de controle.
- Quantos nós (N) e arestas (E) existem?
- Calcule a complexidade ciclomática.
- Liste os caminhos independentes.

Atividade 9

- Considere o código:

```
int processa(int x) {
    if (x > 0) {
        if (x > 10)
            return 1;
        else
            return 2;
    } else {
        return 3;
    }
}
```

- Construa o grafo de fluxo de controle.
- Quantas decisões existem?
- Calcule a complexidade ciclomática.
- Liste todos os caminhos independentes.
- Proponha casos de teste para cobrir todos os caminhos.

Qual dos dois códigos é mais complexo e por quê?

Atividade 10

- Considere o código:

```c
int f(int x) {
    if (x > 0)
        return x;
    else
        return -x;
}

int h(int y) {
    if (y > 5) {
        if (y < 20)
            return 1;
        else
            return 2;
    } else {
        return 3;
    }
}
```

- Construa o grafo de fluxo de controle de cada função.
- Quantos componentes conectados (P) existem?
- Calcule a complexidade ciclomática total.
- Liste os caminhos independentes de cada função.

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **O que é?**
    - Número de níveis de blocos aninhados no código.
    - Inclui estruturas como: **if, for, while, try/catch**.
    - Cada estrutura dentro de outra aumenta a profundidade.
    - Exemplo: **if** dentro de **for** dentro de **if** → profundidade = 3.

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Como surgiu?**
    - Originou-se da necessidade de medir a complexidade do código.
    - Observação de que códigos muito aninhados são mais difíceis de entender.
    - Permite quantificar um aspecto específico da complexidade.
    - Ajuda na análise e refatoração.

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Por que é importante?**
    - Está diretamente ligada à qualidade do código.
    - Menor profundidade → código mais legível.
    - Facilita manutenção e evolução.
    - Reduz risco de erros.
    - Ajuda novos desenvolvedores a entender o sistema mais rápido.
    - Melhora a eficiência do trabalho em equipe.

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Como medir e ajustar?**
    - Medição pode ser:
      - Manual (contando níveis).
      - Automática (ferramentas de análise estática).
    - Para reduzir profundidade:
      - Refatoração
      - Extração de métodos
      - Simplificação de condicionais

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Utilização prática:**
    - Usada em revisão de código.
    - Ajuda a identificar trechos complexos.
    - Serve como guia para refatoração.
    - Útil no ensino para incentivar código limpo desde o início.

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Limitações:**
    - Não mede toda a complexidade do código.
    - Não considera:
      - Tamanho das funções.
      - Acoplamento entre módulos.
    - Deve ser usada junto com outras métricas.

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Limitações:**
    - Não mede toda a complexidade do código.
    - Não considera:
      - Tamanho das funções.
      - Acoplamento entre módulos.
    - Deve ser usada junto com outras métricas.

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Exemplo com problema:**

```
1  public void ProcessOrders(List<Order> orders) {
2      foreach (var order in orders) {
3          if (order.IsValid) {
4              if (order.Payment.IsProcessed) {
5                  foreach (var item in order.Items) {
6                      if (item.InStock) {
7                          // Processar item
8                      }
9                  }
10             }
11         }
12     }
13 }
```

codesnap.dev

Métricas de Complexidade

- **Nesting Depth (Profundidade de Aninhamento):**
  - **Exemplo com problema:**

```
1  public void ProcessOrders(List<Order> orders) {
2      foreach (var order in orders) {
3          ProcessOrder(order);
4      }
5  }
6
7  private void ProcessOrder(Order order) {
8      if (!order.IsValid || !order.Payment.IsProcessed) return;
9      foreach (var item in order.Items) {
10         ProcessItem(item);
11     }
12 }
13
14 private void ProcessItem(Item item) {
15     if (item.InStock) {
16         // Processar item
17     }
18 }
```

codesnap.dev

Atividade 11

- **Qual é a profundidade máxima de aninhamento desse código?**

```
def processa_lista(lista):
    for i in lista:
        if i > 0:
            for j in range(i):
                if j % 2 == 0:
                    print(j)
```

- **Existe uma forma de melhorar a complexidade deste codigo? Se sim, mostre como.**

Métricas de Complexidade

- **Número de parâmetros por função/método:**
  - **Hipótese**: quanto mais parâmetros, maior a complexidade cognitiva e estrutural.
  - **Definição**: Conta o número de parâmetros formais de uma função/método.
  - Para uma função f, a **métrica é**:
    - P(f)= número de parâmetros na assinatura.

Métricas de Complexidade

- **Número de parâmetros por função/método:**
  - Para uma função f, a **métrica** é:
    - P(f)= número de parâmetros na assinatura.

```
def process_data(a, b, c, d):
```

    - P(f) = 4.

Métricas de Complexidade

- **Número de parâmetros por função/método:**
  - **0–2 parâmetros** → baixa complexidade.
  - **3–4 parâmetros** → complexidade moderada.
  - **5 parâmetros** → alta complexidade (code smell).

Métricas de Complexidade

- **Número de parâmetros por função/método:**
  - **Muitos parâmetros** podem indicar:
    - Função faz muitas coisas.
    - Violação de Single Responsibility Principle.
    - Necessidade de refatoração.

Métricas de Complexidade

- **Número de parâmetros por função/método:**
  - **Impacto** no software:
    - ↑Dificuldade de entendimento.
    - ↑ esforço de testes unitários.
    - ↑ probabilidade de bugs de uso (misuse).
    - ↑ acoplamento entre módulos.

Métricas de Complexidade

- **Número de parâmetros por função/método:**
  - **Limitações**:
    - Nem sempre mais parâmetros = mais complexidade real.
    - Funções bem projetadas podem ter muitos parâmetros (ex: APIs).
    - Estruturas como objetos podem reduzir esse número artificialmente.
    - Não captura lógica interna da função.

Métricas de Manuteniblidade

- A **manutenibilidade** não é uma métrica única isolada, mas sim um **resultado combinado de características estruturais do software.**

- Ela é **inferida** a partir de **métricas** de**:**
  - **tamanho** (LOC, pontos de função);
  - **complexidade** (ciclomática, aninhamento, número de parâmetros, métodos/funções).

Métricas de Manuteniblidade

- Como as métricas impactam a manutenibilidade:
  - **Tamanho** (LOC, FP):
    - Código maior → mais difícil de entender e alterar.
    - Aumenta custo de manutenção.
  - **Complexidade** (ciclomática, aninhamento):
    - Mais caminhos de execução → mais esforço de teste e compreensão.
    - Maior risco de defeitos em mudanças.
  - **Estrutura de funções** (parâmetros, métodos):
    - Muitas entradas → maior acoplamento e erro de uso.
    - Muitas funções → maior dispersão da lógica.

Métricas de Manuteniblidade

- O que é “**baixa manutenibilidade**”?
  - Um sistema é considerado de **baixa manutenibilidade** quando apresenta:
    - Alto esforço para entender código.
    - Alto risco de introduzir bugs em mudanças.
    - Dificuldade de teste e refatoração.
    - Forte acoplamento estrutural.

Métricas de Manutenibilidade

- Métricas indiretas de manutenibilidade:
  - **Maintainability Index (MI):**
    - Métrica composta que combina múltiplos fatores estruturais do código.
    - Geralmente inclui:
      - tamanho (LOC);
      - complexidade ciclomática;
      - métricas de Halstead.
    - Interpretação:
      - valores altos → código mais fácil de manter.
      - valores baixos → código potencialmente difícil de evoluir.
    - É uma tentativa de resumir manutenibilidade em um único score.

Métricas de Manuteniblidade

- Métricas indiretas de manutenibilidade:
  - **Code Smells (heurísticas de design):**
    - Padrões no código que sugerem problemas de design.
    - Não são erros, mas sinais de possível degradação da manutenibilidade.
    - Exemplos comuns:
      - funções longas;
      - classes muito grandes (God Class);
      - muitos parâmetros por método;
      - duplicação de código.
    - Indicam aumento do esforço de compreensão e refatoração.

Métricas de Manuteniblidade

- Métricas indiretas de manutenibilidade:
  - **Tendências de crescimento (LOC + complexidade):**
    - Analisa como o sistema evolui ao longo do tempo.
    - Crescimento contínuo de:
      - LOC;
      - complexidade ciclomática.
    - Interpretação:
      - Crescimento descontrolado → provável degradação da arquitetura.
      - Pode indicar acúmulo de dívida técnica.
    - Importante porque manutenibilidade é dinâmica, não estática.

Métricas de Manuteniblidade

- Métricas indiretas de manutenibilidade:
  - **Indicadores de churn (frequência de mudanças):**
    - Mede a quantidade de modificações no código ao longo do tempo.
    - Alta frequência de alterações pode indicar:
      - Código instável;
      - Requisitos mal definidos;
      - Baixa previsibilidade de manutenção.
    - Relação com manutenibilidade:
      - Sistemas com alta manutenibilidade tendem a ter mudanças mais localizadas e seguras.

Dúvidas?